

DOCUMENTACIÓN DEL PROYECTO

k0lmenIA

Agentes de QA para Claude Code: diseño de pruebas, automatización con k0lmena e integración con Xray, QMetry, AIO Tests y Azure DevOps — todo en español.

18 agentes

Web · API · Mobile · Performance

Playwright · Cucumber · Appium · k6 · Artillery

Xray · QMetry · AIO Tests

Jira · Confluence · Figma

PostgreSQL · MySQL · SQL Server · MongoDB

Suites sin tokens

QARMY · Underc0de

github.com/underc0delabs/k0lmenIA

Licencia MIT

Contenido

- 1 Qué es k0lmenIA
- 2 Arquitectura
- 3 Cómo se usa
- 4 Agentes
- 5 Requisitos
- 6 Instalación
- 7 Configuración: el archivo .env
- 8 Investigación de contexto y falta información
- 9 Guía por tarea: diseño y ejecución
- 10 Automatización con k0lmena
- 11 Pruebas de performance
- 12 Gestión de pruebas: Xray, QMetry y AIO Tests
- 13 Verificación en base de datos
- 14 Conectores MCP
- 15 Convenciones
- 16 Estructura del repositorio
- 17 Problemas frecuentes
- 18 Extender el proyecto

INTRODUCCIÓN

Qué es k0lmenIA

Un equipo de **agentes especializados en QA** para Claude Code, pensado para profesionales de testing manual: no hace falta programar, se trabaja conversando dentro de VS Code.

Ponés un insumo (una historia, una observación de bug, un contrato de API, o directamente una key de Jira o un link de Figma), pedís lo que necesitás en lenguaje natural y el agente que corresponde genera el resultado: análisis, planes, casos en Excel o Gherkin, datos, bugs, reportes HTML, automatización y carga en la herramienta de gestión.



1 · Diseño

Analizar historias, planificar, escribir casos manuales, BDD y de API, generar datos y redactar bugs.

Queda en `output/`



2 · Automatización

Los agentes *mapper* escriben la automatización una sola vez; después corre con `npm`, sin tokens.

Queda en `herramientas/k0lmena/`



3 · Gestión

Casos, ciclos y resultados con evidencias en Xray, QMetry o AIO Tests (casos también en Azure DevOps), y el informe de cierre.

Tu herramienta + `output/`

Principios

PRINCIPIO	EN LA PRÁCTICA
No inventar	Si falta un paso, un dato, una regla o un umbral, el agente lo marca y lo pregunta. Mejor un artefacto con huecos señalados que uno completo pero inventado.
Respetar los formatos	Los campos, su orden y sus valores los definen las plantillas y los scripts, no el agente.
Trazabilidad	Todo referencia la historia (<code>HU-XXX</code>); cada escenario automatizado lleva el ID del caso (<code>@CP-001</code>) y la key de la herramienta de gestión.
Cobertura pensada	Positivos, negativos, bordes y validaciones de campos, no solo el camino feliz.
Investigar antes de suponer	Antes de trabajar una historia se revisan sus comentarios, subtareas, épica, issues vinculados, Confluence y Figma. Lo que no aparece queda marcado como Falta información (<code>FI-01</code>).
Ahorro de tokens	Lo que se repite se automatiza una vez y después corre sin agentes.
Seguridad	Los secretos viven en el <code>.env</code> de la raíz, que nunca se versiona; los agentes nunca piden tokens por el chat.



Arquitectura de k0lmenIA · [hacé clic para ver en tamaño completo](#)

PIEZA	QUÉ ES	DÓNDE
Claude Code	Orquestador: interpreta el pedido, elige el agente y aplica los estándares del proyecto.	<code>CLAUDE.md</code>
Agentes	El "quién": un especialista por tarea.	<code>.claude/agents/</code>
Skills	El "cómo": conocimiento que se carga cuando hace falta (técnicas de diseño, ejecución E2E y de API, convenciones de k0lmena).	<code>.claude/skills/</code>
Plantillas y scripts	Formato de salida (planilla de casos, cobertura, bug) y reportes HTML determinísticos.	<code>plantillas/</code> · <code>scripts/</code>
Integración de gestión	<code>gestion.py</code> con un adaptador por herramienta: Xray Cloud, Xray Server/DC, QTM4J y AIO Tests. Azure DevOps va por su conector MCP.	<code>scripts/gestion/</code>
Conectores MCP	Playwright (activo), Jira y Confluence, Figma, Appium, QMettry, AIO Tests, k0lmenaTMT y Azure DevOps.	<code>.mcp.json</code>
Herramientas	Newman para colecciones de Postman y k0lmena para web, API, mobile y performance.	<code>herramientas/</code>
Entrada y salida	Insumos del usuario y artefactos generados, ordenados por tipo.	<code>input/</code> · <code>output/</code>

k0lmenIA · Cómo se usa

Le pedís a Claude Code lo que necesitás en lenguaje natural; el agente que corresponde hace el trabajo y deja el resultado

QARMY · UNDERCODE

1 Diseño

De la historia a los casos de prueba

0 **Poné la historia en `input/historias/`**
o pasá una key de Jira: los agentes investigan comentarios, subtareas, épica, Confluence y Figma, arman la ficha de contexto y marcan lo que falta (FI-01)

1 **"Analizá la historia HU-001"**
analista-historias
→ ambigüedades y preguntas para el PO

2 **"Armá el plan de pruebas de HU-001"**
estratega-pruebas
→ plan HTML: alcance, riesgos, criterios

3 **"Generá los casos de HU-001" · "Pasala a BDD"**
generador-casos-manuales · generador-casos-bdd · generador-casos-api
→ casos-HU-001.xlsx + .md + cobertura · .feature

4 **"Generá datos de prueba" · "Armá el reporte de este bug"**
generador-datos-prueba · generador-reportes-bug
→ datos CSV/MD · BUG-001.md

UNA SOLA CONFIGURACIÓN

El `.env` de la raíz: URLs, credenciales, k0lmena y gestión. Nunca se commitea.

NO INVENTA

Busca en Jira, Confluence y Figma; lo que no encuentra queda como **Falta información** (FI-01) en todos los reportes.

AHORRA TOKENS

Lo que se repite se automatiza una vez y corre con `npm`, sin agentes.

TRAZABILIDAD

HU-001 → CP-001 → @CP-001 → key en Xray/QMetry/AIO → resultado.

2 Ejecución y automatización

Una vez con agentes, después sin tokens

5 **"Automatizá en k0lmena los casos de HU-001 contra `https://tu-app.com`"**
web-mapper · api-mapper (Postman / Swagger) · mobile-mapper (Appium) · verificador-datos (base de datos)
→ recorre la app, ajusta los pasos y escribe `.feature` + steps + locators **UNA VEZ**

6 **Corrés la suite cuando quieras **SIN TOKENS****

```
cd herramientas/k0lmena
npm test # web + api (TAGS=@HU-001)
npm run test:mobile
npm run report:web # HTML con evidencias
```

7 **"Armá una prueba de carga del login para 50 usuarios"**
performance-mapper: pregunta ambiente, carga y umbrales · k6 o Artillery

```
npm run perf -- HU-001-login smoke
npm run perf -- HU-001-login load # pide confirmación
```

¿Una prueba puntual que no vale la pena automatizar? → "Ejecutá el escenario X contra `https://...`" (ejecutor-e2e / ejecutor-api, en vivo)

3 Gestión y cierre

Xray · QMetry (QTM4J) · AIO Tests

8 **"Subí los casos de HU-001 a Xray en 'HU-001 Registro', vinculados a PROJ-12"**
gestor-pruebas
→ carpeta + casos vinculados a la historia

9 **"Creá el ciclo Sprint 5 con esos casos"**
gestor-pruebas
→ ciclo (Test Execution) vinculado a PROJ-12

10 **"Subí los resultados de la última corrida al ciclo PROJ-60"**
publicador-resultados
→ estado + error + captura, GIF y video (si falló) por caso

11 **"Armá el informe de cierre de HU-001"**
generador-reportes-cierre
→ resultados, bugs, riesgos y go / no-go

Flujo de punta a punta · hacé clic para ver en tamaño completo

- 1

Poné tus insumos
En `input/`, o pasá una key de Jira, una página de Confluence o un link de Figma con el conector activo. Los agentes investigan el contexto completo y marcan lo que falta (`FI-01`).
- 2

Pedí en lenguaje natural
Claude Code elige el agente. También podés nombrarlo: *"usá el web-mapper para..."*.
- 3

Revisá el resultado
En `output/` o en `herramientas/k0lmena/`.
- 4

Automatizá lo que se repite
Y corrélo con `npm test` todas las veces que quieras, sin tokens.

#	LE PEDÍS	OBTENÉS
0	"Investigá el contexto de PROJ-12"	Ficha de contexto + Falta información
1	"Analizá la historia HU-001"	Ambigüedades y preguntas para el PO
2	"Armá el plan de pruebas de HU-001"	Plan HTML
3	"Generá los casos de HU-001"	<code>casos-HU-001.xlsx</code> + cobertura
4	"Subí los casos a Xray en 'HU-001 Registro', vinculados a PROJ-12, y creá el ciclo Sprint 5"	Casos y ciclo en Xray
5	"Automatizá en k0lmena los casos de HU-001 contra https://tu-app.com"	<code>.feature</code> + steps + locators
6	<code>npm test</code> · <code>npm run report:web</code>	Reporte HTML con evidencias
7	"Subí los resultados al ciclo del Sprint 5"	Estados y evidencias en Xray
8	"Armá una prueba de carga del login para 20 usuarios"	Script k6/Artillery + reporte HTML
9	"Armá el informe de cierre de HU-001"	Go / no-go

INTRODUCCIÓN

Agentes

Claude Code elige el agente según lo que pidas. Están en `.claude/agents/` y se agrupan en cuatro familias.

Análisis y diseño

analista-historias

Investiga el contexto de la historia y detecta ambigüedades, vacíos, riesgos y Falta información.

`output/analisis-historias/`

estrategia-pruebas

Plan de pruebas: alcance, riesgos, tipos de prueba, entorno y criterios, en HTML.

`output/planes-de-prueba/`

generador-casos-manuales

Casos en Excel y Markdown + informe de cobertura con preguntas para el PO.

`output/casos-de-prueba/manuales/`

generador-casos-bdd

Escenarios Gherkin (keywords en inglés, contenido en español) + cobertura.

`output/casos-de-prueba/bdd/`

generador-casos-api

Casos de API positivos, negativos, de schema y de autorización.

`output/casos-api/`

generador-datos-prueba

Datos válidos, inválidos y de borde en Markdown o CSV.

`output/datos-de-prueba/`

generador-reportes-bug

Reporte de bug profesional según la plantilla, con severidad y prioridad justificadas.

`output/reportes-bug/`

Ejecución en vivo USA TOKENS EN CADA CORRIDA

ejecutor-e2e

Ejecuta casos en un navegador real con Playwright MCP (headed o headless), con evidencia.

`output/ejecuciones/`

ejecutor-api

Corre una colección de Postman con Newman.

`output/ejecuciones/`

generador-reporte-html

Dashboard HTML de los resultados de una ejecución.

`output/ejecuciones/`

generador-reporte-cierre

Informe de cierre de la ronda con recomendación go/no-go.

`output/informes-cierre/`

Automatización con k0lmena DESPUÉS CORRE SIN TOKENS

web-mapper

Recorre la web siguiendo tus casos y genera `.feature`, steps y locators.

`k0lmena/web/`

api-mapper

Lee Postman o Swagger/OpenAPI, verifica los endpoints y genera los `.feature`.

`k0lmena/api/`

mobile-mapper

Recorre la app con Appium y genera `.feature`, steps y locators.

`k0lmena/mobile/`

performance-mapper

Carga, estrés, soak y picos con k6 o Artillery; pregunta carga y umbrales.

`k0lmena/performance/`

verificador-datos

Verifica datos en PostgreSQL, MySQL, SQL Server o MongoDB y suma verificaciones de BD a los tests.

Chat · steps en

`.feature`

Gestión de pruebas

gestor-pruebas

Carpetas, casos, vínculos con la historia y ciclos en Xray, QMetry (QTM4J), AIO Tests o Azure DevOps Test Plans.

La herramienta +

`output/gestion/`

publicador-resultados

Sube los resultados de `npm test` al ciclo: estado, error y evidencias.

La herramienta

Requisitos

Instalá solo lo que vayas a usar.

PARA...	NECESITÁS
Usar los agentes	Claude Code y una cuenta de Claude (Pro, Max, Team o Enterprise) o acceso por API de Anthropic. VS Code recomendado.
Casos, reportes y gestión	Python 3 con <code>openpyxl</code> , <code>tabulate</code> y <code>requests</code> .
Ejecutar E2E en vivo	Node.js 18+ y los navegadores de Playwright.
Ejecutar colecciones de Postman	Newman .
Automatizar con k0lmena	Node.js 20+ .
Automatizar mobile	Node.js 22+ , JDK y Android SDK, o macOS con Xcode para iOS; o una cuenta de BrowserStack.

Instalación

En cinco minutos tenés los agentes andando; k0lmena y el resto se suman cuando los necesites.

1

Instalá Claude Code

```
curl -fsSL https://claude.ai/install.sh | bash      # macOS / Linux
npm install -g @anthropic-ai/claude-code           # alternativa (Node.js 18+)
claude --version
```

En Windows, seguí la [guía oficial](#).

2

Cloná el repositorio

```
git clone https://github.com/underc0delabs/k0lmenaIA.git
cd k0lmenaIA
pip install -r requirements.txt
cp .env.example .env                                # y completalo
```

3

Abrí VS Code y lanzá Claude Code

```
code .
claude                                             # la primera vez pide autenticarte
```

4

Sumá lo opcional

```
npx playwright install                        # ejecución E2E en vivo
npm install -g newman                        # colecciones de Postman
npm install                                  # k0lmena (desde la raíz)
cd herramientas/k0lmena
npx playwright install chromium
npm run bootstrap:k6                          # performance de APIs con k6
```

Configuración: el archivo .env

Hay **un solo** `.env`, **en la raíz del repo**. Lo usan los agentes, k0lmena y los scripts de gestión. Está en `.gitignore` y la plantilla comentada es `.env.example`.

GRUPO	VARIABLES	PARA QUÉ
App bajo prueba	<code>APP_URL</code> <code>APP_USER</code> <code>APP_PASSWORD</code>	Ejecución E2E y logins
API	<code>API_TOKEN</code> <code>API_BASEURL</code>	Newman, suite de API y k6
Web	<code>BASEURL</code> <code>BROWSER</code> <code>HEADLESS</code> <code>VIEWPORT_WIDTH</code> <code>VIEWPORT_HEIGHT</code> <code>LOCALE</code> <code>TIMEZONE</code>	Navegador de la suite web
Ejecución	<code>TAGS</code> <code>PARALLEL</code>	Qué escenarios y cómo se corren
Evidencias	<code>EVIDENCE</code> (<code>captura</code> · <code>ambos</code> · <code>off</code>) <code>VIDEO</code> <code>TRACE</code>	Qué se adjunta al reporte
Auto-healing	<code>KOLMENA_AUTO_HEALING</code>	Recuperar locators rotos
Mobile	<code>MOBILE_TARGET</code> <code>MOBILE_PLATFORM</code> <code>MOBILE_DEVICE_NAME</code> <code>MOBILE_APP</code> <code>MOBILE_UDID</code> <code>BROWSERSTACK_*</code>	Dónde corre la suite mobile
Performance	<code>PERF_VUS</code> <code>PERF_DURACION</code>	Pisar la carga de los scripts k6
Bases de datos	<code>DB_CONEXIONES</code> + <code>DB_<NOMBRE>_MOTOR</code> <code>_HOST</code> <code>_PUERTO</code> <code>_BASE</code> <code>_USUARIO</code> <code>_CLAVE</code> (o <code>_URL</code>) <code>_ESCRITURA</code> <code>_PRODUCCION</code>	Verificación en base de datos
Gestión	<code>GESTION_HERRAMIENTA</code> <code>GESTION_PROYECTO</code> + credenciales de la herramienta	Xray, QMetry y AIO Tests
Azure DevOps (MCP)	<code>ADO_ORGANIZACION</code> <code>ADO_PAT</code> <code>ADO_PROYECTO</code>	Boards, Wiki y Test Plans

! Usá credenciales de un entorno de prueba, nunca de producción. Los agentes leen los secretos del `.env`: nunca pegues un token en el chat ni lo escribas en un archivo versionado.

Carpetas de entrada

CARPETA	QUÉ VA
<code>input/historias/</code>	Historias de usuario con sus criterios de aceptación (<code>HU-001.md</code> , ...).
<code>input/documentacion/</code>	Documentación del producto que sirva de contexto.
<code>input/api/</code>	Contratos, colecciones de Postman y environments.
<code>input/bugs/</code>	Observaciones sueltas de errores para convertir en reportes.

Investigación de contexto y "Falta información"

Antes de analizar, planificar, escribir casos o automatizar, los agentes **investigan el contexto completo** de la historia en lugar de suponer. Lo que no encuentran en ninguna fuente queda marcado como **Falta información** en todos los reportes.

Dónde buscan

FUENTE	QUÉ REVISAN
Historia de Jira	Descripción, criterios, estado, labels, componentes y adjuntos.
Comentarios	Todos. Las decisiones que se tomaron después ("acordamos que el límite es 50") reemplazan a la descripción, citando quién y cuándo.
Subtareas, épica e issues vinculados	Detalle de cada subtarea, reglas generales e historias hermanas de la épica, dependencias y bugs conocidos del flujo.
Confluence	Páginas enlazadas o, si no hay, las más relevantes del espacio del proyecto.
Figma	Textos exactos, mensajes de error, estados y campos obligatorios.
Contratos e <code>input/</code>	Swagger, Postman y documentación local.

El resultado es una **ficha de contexto** reutilizable, `output/contexto/contexto-HU-XXX.md` : fuentes consultadas, hallazgos por criterio, decisiones, contradicciones y lo que falta. Si la historia no cambió, los demás agentes la reusan sin volver a investigar. Si un conector no está activo, la fuente queda como "No disponible" en lugar de suponerla.

PEDIDO · *Investigá el contexto de PROJ-12 y generá los casos de prueba.*

Falta información (FI-XX)

Cada dato que no aparece en ninguna fuente (o aparece contradictorio) recibe un ID por historia, con qué falta, dónde se buscó, a qué afecta y la pregunta para el PO. El mismo ID viaja a todos los artefactos. La app no es fuente de requisitos: si el resultado esperado depende de un dato faltante, el escenario no se "acomoda" a lo que hace la aplicación; se automatiza con el supuesto marcado o queda `@bloqueado` hasta que el PO responda.

DÓNDE	CÓMO SE VE
Planilla de casos	Casos resaltados en ámbar, etiqueta <code>@falta-info</code> , detalle en Comentarios y hoja Falta información .
Análisis y coberturas	Sección Falta información ; criterios con cobertura <code>Parcial (falta información)</code> .
<code>.feature</code> de k0lmena	Tags <code>@falta-info @FI-01</code> y comentario <code># FALTA INFORMACIÓN (FI-01): ...</code> .
Reporte de mapeo	Estado <code>Falta información</code> . El mapper no usa la app para completar el dato: lo que muestra la app queda como observación.
Reportes de k0lmena y de ejecución	Nota visible en el escenario y aviso en el resumen.
Informe de cierre	Sección Falta información ; los ítems abiertos cuentan como riesgo para el go/no-go.
Xray · QMetry · AIO	El comentario del resultado aclara que el caso depende de información faltante.

Diseño y ejecución

Qué pedir, qué hace cada agente y qué deja.

Analizar una historia

ANALISTA-HISTORIAS

PEDIDO · Analizá la historia HU-001 y decime qué ambigüedades tiene.

Revisa cada criterio y entrega `analisis-HU-001.md` con ambigüedades, vacíos, riesgos, casos no contemplados y preguntas para el PO.

Armar el plan de pruebas

ESTRATEGA-PRUEBAS

PEDIDO · Armá el plan de pruebas de HU-001.

Dashboard HTML con alcance, objetivos, tipos de prueba, riesgos, datos y entorno, y criterios de entrada y salida.

Generar casos manuales

GENERADOR-CASOS-MANUALES

PEDIDO · Generá los casos de prueba de HU-001.

Entrega `casos-HU-001.xlsx`, `casos-HU-001.md` y `casos-HU-001-cobertura.md`. Cubre camino feliz, negativos, bordes y validaciones. Esa planilla es la que después se sube a la herramienta de gestión o se automatiza.

Generar escenarios BDD

GENERADOR-CASOS-BDD

PEDIDO · Pasá la historia HU-001 a escenarios BDD.

Genera `HU-001-<slug>.feature`, con un tag `@CP-XXX` por escenario, y `HU-001-cobertura.md`.

Generar casos de API

GENERADOR-CASOS-API

PEDIDO · Generá los casos de la API de autenticación a partir de `input/api/auth-endpoints.md`.

Tabla resumen y detalle por caso: método, endpoint, headers, body, status y schema esperado.

Datos de prueba y bugs

GENERADOR-DATOS-PRUEBA

GENERADOR-REPORTES-BUG

PEDIDO · Generá datos de prueba para el formulario de registro, en CSV.

Datos válidos, inválidos y de borde. Los bugs siguen `plantillas/plantilla-reporte-bug.md` (editala para usar tu formato) y marcan lo que falta en vez de inventarlo.

Ejecutar en vivo

[EJECUTOR-E2E](#)[EJECUTOR-API](#)[USA TOKENS](#)

PEDIDO · Ejecutá *SOLO* el escenario de registro válido de HU-001 contra <https://tu-app.com>.

El ejecutor E2E pregunta si querés ver el navegador, ejecuta con Playwright MCP, saca evidencia y genera `reporte-HU-001-<fecha-hora>.html`. El de API corre la colección con Newman. Ideal para pruebas puntuales: lo que se repite conviene automatizarlo.

Reportes e informe de cierre

[GENERADOR-REPORTE-HTML](#)[GENERADOR-REPORTE-CIERRE](#)

PEDIDO · Armá el informe de cierre de las pruebas de HU-001.

Dashboards con indicadores, gráficos y detalle; el informe de cierre suma bugs abiertos, riesgos y la recomendación go/no-go.

Automatización con k0lmena

El framework integrado en `herramientas/k0lmena/`. Un agente *mapper* recorre la aplicación **una sola vez** y escribe la automatización; desde ahí la suite corre con `npm`, sin agentes y sin tokens.



Web

Playwright + Cucumber, video, trace y GIF. Con `K0LMENA_AUTO_HEALING=on` recupera locators rotos.



API

axios + Cucumber con steps genéricos en español: no hace falta programar.



Mobile

WebdriverIO + Appium en dispositivo, emulador o BrowserStack.



Performance

k6 para APIs y Artillery + Playwright para flujos web.

- i Las carpetas de k0lmena vienen vacías: todo lo que hay adentro lo generan los mappers para tu aplicación.

web-mapper

PEDIDO · Automatizá en k0lmena los casos de output/casos-de-prueba/manuales/casos-HU-001.xlsx contra `https://tu-app.com`.

1. Lee los casos (manuales o BDD).
2. Navega la web real con Playwright MCP y comprueba que cada paso se puede ejecutar.
3. Genera `web/features/HU-001-<slug>.feature`, `web/steps/HU-001.steps.ts` y `web/locators/HU-001.locators.ts`, reutilizando steps existentes.
4. Valida con `npm run test:web` y deja `output/mapeos/mapeo-HU-001-web.md`.

Si un paso no se puede ejecutar, el escenario queda `@bloqueado` con el motivo y no corre hasta que se resuelva.

api-mapper

PEDIDO · Pasá a k0lmena los endpoints de /pet del Swagger `https://petstore.swagger.io/v2/swagger.json`.

Lee Postman (v2.x) o Swagger/OpenAPI, te deja elegir los endpoints, diseña escenarios positivos, negativos y de autorización, los verifica contra la API real y escribe los `.feature` con los steps genéricos. Antes de un POST, PUT, PATCH o DELETE pide autorización.

mobile-mapper

PEDIDO · Automatizá en k0lmena los casos de login de la app en el emulador.

Necesita el conector Appium MCP y la app en `mobile/apps/` (no se versiona). Después la suite corre sin MCP según `MOBILE_TARGET`:

DESTINO	QUÉ NECESITÁS
<code>device</code>	Dispositivo por USB con depuración y <code>MOBILE_UDID</code> (de <code>adb devices</code>).
<code>emulator</code>	Emulador o simulador encendido y <code>MOBILE_DEVICE_NAME</code> .
<code>browserstack</code>	<code>BROWSERSTACK_USER</code> , <code>BROWSERSTACK_KEY</code> y <code>BROWSERSTACK_APP</code> .

Correr la suite

```
cd herramientas/k0lmena
npm test # web + API
npm run test:web # también: test:api · test:mobile · test:all
TAGS=@HU-001 npm test # filtrar por tag (bash)
$env:TAGS="@HU-001"; npm test # filtrar por tag (PowerShell)
npm run report:web # también: report:api · report:mobile
```

Si `EVIDENCE` está vacía, `npm test` pregunta al arrancar qué guardar de los tests que pasan: solo captura, o captura + GIF del recorrido.

Evidencias en el reporte

SUITE	ESCENARIO QUE PASA	ESCENARIO QUE FALLA
Web	Captura final (+ GIF opcional)	Video completo, captura, error con stack, URL y título, logs del navegador (consola, errores JS, requests fallidos, HTTP $\geq$ 400), logs de Node y trace de Playwright
API	Request y response (el <code>Authorization</code> se oculta)	Lo mismo, más la validación que falló
Mobile	Captura final (+ GIF opcional)	Video completo, captura, page source y logs del dispositivo

Herramientas extra

COMANDO	PARA QUÉ
<code>npm run crawler [url]</code>	Genera un archivo de locators recorriendo una página.
<code>npm run link-tester</code>	Busca enlaces e imágenes rotas en <code>BASEURL</code> .
<code>npm run record</code>	Graba un flujo con Playwright codegen.
<code>npm run debug</code>	Corre la suite web en modo debug.

Pruebas de performance

El **performance-mapper** arma pruebas de carga, estrés, soak y picos. Nunca usa valores por defecto: la carga y los umbrales los definís vos.

! Una prueba de carga genera tráfico real. Corredla solo contra ambientes de prueba o con autorización de los dueños del sistema.

performance-mapper

PEDIDO · Armá una prueba de carga de los endpoints de /pet para 50 usuarios.

1. **Pregunta** el ambiente (y que no sea producción), el flujo o los endpoints, la carga objetivo, los umbrales (p95, p99, % de error), los perfiles y si puede usar métodos con efecto.
2. **Elige la herramienta**: k6 para APIs (escala a miles de usuarios) o Artillery + Playwright para flujos de navegador (pocos usuarios: cada uno es un Chromium).
3. **Escribe el script** una vez y lo valida con un smoke.
4. **Corre con carga** solo con tu confirmación explícita para cada corrida.

Comandos

```
npm run perf # lista los scripts
npm run perf -- <script> smoke # valida el script con carga mínima
npm run perf -- <script> load # carga objetivo (pide confirmación)
npm run perf -- <script> stress --confirmar # sin pregunta: CI o con autorización
npm run perf -- <script> load --vus 20 --duracion 2m # k6: pisa la carga
```

PERFIL	QUÉ HACE
smoke	Carga mínima para validar que el script funciona.
load	Sube hasta la carga objetivo, la sostiene y baja.
stress	1x, 2x y 3x la carga objetivo: busca el punto de quiebre.
soak	Carga objetivo durante mucho tiempo: fugas de memoria y degradación.
spike	Salto brusco a un pico y vuelta: cómo se recupera.

Cada corrida deja en `reports/performance/<k6|artillery>/` un reporte HTML con el veredicto, requests, req/s, errores, latencias p50/p95/p99, umbrales con el valor medido, detalle por endpoint o paso, códigos de respuesta, checks y, en Artillery, la evolución en el tiempo. Si un umbral no se cumple, el comando termina con error.

Xray, QMetry y AIO Tests

gestor-pruebas y **publicador-resultados** trabajan con Xray Cloud, Xray Server/Data Center, QMetry para Jira (QTM4J) y AIO Tests, con los mismos comandos para todas.

Se configura en el `.env`: `GESTION_HERRAMIENTA` (`xray-cloud`, `xray-dc`, `qtm4j` o `aio`), `GESTION_PROYECTO` (la key del proyecto de Jira) y las credenciales de tu herramienta, explicadas en `.env.example`.

gestor-pruebas

PEDIDO · Subí los casos de HU-001 a Xray en la carpeta 'HU-001 Registro', vinculados a PROJ-12, y creá el ciclo Sprint 5.

Crea la carpeta, sube los casos (manuales desde el `.xlsx` o BDD desde el `.feature`), los vincula a la historia, crea ciclos y les agrega casos. La primera vez hace una prueba en seco (`--dry-run`). Guarda la trazabilidad en `output/gestion/HU-001-<herramienta>.json`: un caso nunca se sube dos veces.

publicador-resultados

PEDIDO · Subí los resultados de la última corrida web al ciclo PROJ-60.

Asocia cada escenario a su caso por el tag `@CP-XXX` y carga el estado, un comentario con el error y las evidencias: captura siempre, GIF si se generó y video solo si falló. Nunca asigna un resultado por aproximación.

Equivalencias

CONCEPTO	XRAY	QTM4J	AIO TESTS
Caso	Issue Test (Manual o Cucumber)	Test case	Caso (Classic o BDD)
Ciclo	Test Execution	Test cycle	Test cycle
Caso ↔ historia	Link "Test" (cobertura)	Requirement link	jiraRequirementIDs
Ciclo ↔ historia	Link "Relates"	Requirement link	jiraTaskIDs

Comandos manuales

```
python scripts/gestion/gestion.py --dry-run carpeta --ruta "HU-001 Registro"
python scripts/gestion/gestion.py subir-casos --origen <casos.xlsx> --carpeta "HU-001 Registro" --historia PROJ-12 --traza HU-001
python scripts/gestion/gestion.py crear-ciclo --nombre "Sprint 5" --historia PROJ-12 --traza HU-001
python scripts/gestion/gestion.py extraer-resultados --reporte <cucumber-report.json>
python scripts/gestion/gestion.py publicar-resultados --ciclo PROJ-60 --resultados <resultados.json> --traza HU-001
```

Las carpetas van sin barra inicial (en Git Bash, `/HU-001` se convierte en una ruta de Windows). Referencia completa en `scripts/gestion/README.md`.

Azure DevOps (Test Plans)

Azure DevOps no pasa por `gestion.py`: se integra por su conector MCP oficial (`azure-devops`), con `ADO_ORGANIZACION` , `ADO_PAT` y `ADO_PROYECTO` en el `.env` . Los agentes leen las historias (con comentarios, tareas, padre y vínculos) y la Wiki, y **gestor-pruebas** crea los *Test Cases* con sus pasos (los arma `scripts/gestion/para_azure_devops.py`), los vincula a la historia y los agrega a una suite. Todavía no se publican resultados de ejecución en Azure DevOps.

PEDIDO · *Subí los casos de casos-HU-001.xlsx a Azure Test Plans, plan 'Sprint 5', suite 'HU-001 Registro', vinculados a la historia 1234.*

Verificación en base de datos

El **verificador-datos** revisa lo que las pruebas no ven: qué quedó guardado, en qué estado y con qué valores. Soporta **PostgreSQL, MySQL / MariaDB, SQL Server y MongoDB**, con varias conexiones con nombre.



A pedido

"Verificá si existe el usuario ana@test.com y en qué estado quedó." Mira el esquema, arma la consulta con parámetros y te responde.



Dentro de los tests, si lo pedís

Suma steps de base de datos a los `.feature` de web, API o mobile solo cuando se lo pedís; después corren con `npm test`, sin tokens.



Seguro por defecto

Solo lectura, en transacciones que se descartan; datos sensibles enmascarados. Escribir requiere habilitarlo y confirmar cada operación.

Configuración

```
DB_CONEXIONES=principal,pagos           # la primera es la de por defecto
DB_PRINCIPAL_MOTOR=postgres              # postgres | mysql | mariadb | sqlserver | mongodb
DB_PRINCIPAL_HOST=localhost
DB_PRINCIPAL_PUERTO=5432
DB_PRINCIPAL_BASE=tienda
DB_PRINCIPAL_USUARIO=qa_lectura
DB_PRINCIPAL_CLAVE=...
DB_PRINCIPAL_ESCRITURA=no                # si = permite escribir con confirmación
DB_PRINCIPAL_PRODUCCION=no                # si = nunca escribe
```

Steps para los tests

i Solo cuando lo pedís. Los mappers (web, API y mobile) no agregan verificaciones de base de datos por su cuenta: si un caso menciona datos guardados, lo dejan como sugerencia en el reporte de mapeo. El verificador-datos, ante una consulta puntual, responde sin tocar los `.feature`.

```
When envío un POST a "/usuarios" con el body: ...
And guardo el campo "id" de la respuesta como "idUserario"
Then existe en la base de datos un registro en "usuarios" donde "id" es "{idUserario}"
And el registro de "usuarios" donde "id" es "{idUserario}" tiene:
  | campo | valor |
  | estado | ACTIVO |
  | baja | (nulo) |
And la base de datos "pagos" tiene 1 registro en "movimientos" donde:
  | campo | valor |
  | usuario_id | {idUserario} |
```

También: `no existe en la base de datos ...`, `consulto en la base de datos:` (SQL, o JSON en MongoDB), `la consulta devuelve N filas`, `el campo "x" de la consulta es "y"` y `guardo el campo "x" de la consulta como "var"`. Si el dato se guarda de forma asíncrona, los steps reintentan hasta `DB_ESPERA_MS`. Cada verificación deja en el reporte la consulta y las filas encontradas.

Sin agente

```
cd herramientas/k0lmena
npm run bd -- conexiones                # lista y prueba las conexiones
npm run bd -- esquema --tabla usuarios  # columnas de una tabla
npm run bd -- existe --tabla usuarios --donde email=ana@test.com
npm run bd -- consultar --sql "SELECT id, estado FROM usuarios WHERE email = ?" --param ana@test.com
```

! Para preparar o limpiar datos, la conexión necesita `DB_<NOMBRE>_ESCRITURA=si` y el agente muestra la sentencia y pide confirmación antes de cada operación. Con `DB_<NOMBRE>_PRODUCCION=si` nunca escribe. Recomendado: un usuario de base de datos con permisos de solo lectura.

Conectores MCP

Conectan a Claude Code con sistemas externos. Todos están en `.mcp.json`, sin secretos: las credenciales se leen del `.env`. Solo Playwright viene activo.

CONECTOR	PARA QUÉ	AUTENTICACIÓN
<code>playwright</code>	Navegador para el ejecutor E2E y el web-mapper (headed y headless).	Activo
<code>atlassian</code>	Jira y Confluence: historias, criterios y documentación funcional directamente.	Cuenta (OAuth)
<code>figma</code>	Diseños desde un link a un frame o archivo: textos, estados y componentes. Requiere asiento Full o Dev.	Cuenta (OAuth)
<code>appium-mcp</code>	Lo usa el mobile-mapper para recorrer la app.	<code>ANDROID_ID_HOME</code>
<code>qmetry</code> · <code>qtm4j</code>	Consultar QMetry desde el chat.	API key
<code>aio-tests</code>	Consultar AIO Tests desde el chat.	Token
<code>azure-devops</code>	Azure DevOps: historias, Wiki y casos en Test Plans.	PAT
<code>k0lmena-tmt</code>	Cargar y consultar casos en k0lmenaTMT.	Token personal

- 1

Completá el .env
Si usa token, cargalo en el `.env` de la raíz. Atlassian y Figma se autorizan con tu cuenta desde `/mcp`.
- 2

Habilitalo
Agregá su nombre a `enabledMcpjsonServers` en `.claude/settings.local.json`.
- 3

Verificá
Reiniciá Claude Code y revisá la conexión con `/mcp`.

PEDIDO · *Generá los casos de PROJ-12 usando la página de Confluence 'Reglas de facturación' y este diseño de Figma.*

Para Xray no hace falta MCP: se usa `scripts/gestion/`. Guía completa en `CONECTORES.md`.

Convenciones

ELEMENTO	FORMATO
Historias	HU-001
Criterios de aceptación	CA1 · CA2
Casos manuales	CP-001
Casos de API	CP-API-001
Bugs	BUG-001
Falta información	FI-01 · @falta-info @FI-01

ESCALA	VALORES
Severidad y prioridad	Crítica · Alta · Media · Baja
Estado de un caso	N/A · Pendiente · En ejecución · Aprobado · Fallido · Bloqueado

Tags en k0lmena

DÓNDE	TAGS
Feature	La historia y el tipo: @HU-001 @web · @api · @mobile
Scenario	El ID del caso (@CP-001 , @CP-API-001) y @Smoke si es crítico
Excluir	@bloqueado : un paso no se pudo verificar y el escenario no se ejecuta

Nombres de archivos

ARTEFACTO	NOMBRE
Análisis	analisis-HU-001.md
Casos manuales	casos-HU-001.xlsx + .md + -cobertura.md
Casos BDD	HU-001-registro.feature + HU-001-cobertura.md
Bug	BUG-001.md
Automatización	<tipo>/features/HU-001-<slug>.feature · steps/HU-001.steps.ts · locators/HU-001.locators.ts
Performance	performance/k6/http/HU-001-<slug>.ts · performance/artillery/HU-001-<slug>.yaml
Mapeo	output/mapeos/mapeo-HU-001-<tipo>.md

Estructura del repositorio

```

k0lmenIA/
├─ CLAUDE.md           Contexto y estándares (Claude Code lo lee siempre)
├─ ARQUITECTURA.md    Cómo crece el repo
├─ CONECTORES.md       Cómo activar los conectores MCP
├─ .env.example         Plantilla de variables (copiar a .env)
├─ .mcp.json            Conectores MCP (sin secretos; activos según settings)
├─ package.json         Atajos para correr k0lmena desde la raíz (npm test...)
├─ docs/               Este sitio, el PDF y los diagramas
├─ .claude/
│   └─ agents/          Los 18 agentes
│   └─ skills/          Contexto, diseño, ejecución E2E y API, k0lmena
├─ input/              historias/ documentacion/ api/ bugs/
├─ output/             Lo que generan los agentes
├─ plantillas/         Bug, planilla de casos, cobertura
├─ scripts/            Casos, reportes HTML, plan, informe de cierre
│   └─ gestion/         Xray, QMetry (QTM4J), AIO Tests y casos para Azure DevOps
│   └─ mcp/             Lanzadores de los conectores MCP (leen el .env)
├─ herramientas/
│   └─ newman/          Colecciones de Postman
│   └─ k0lmena/
│       └─ web/         features/ steps/ locators/ + framework
│       └─ api/         features/ steps/ (steps genéricos)
│       └─ mobile/      features/ steps/ locators/ support/ apps/
│       └─ performance/ k6/http/ y artillery/
│       └─ reports/     Reportes HTML
│       └─ run-tests.js  Runner de npm test
│       └─ run-perf.js   Runner de npm run perf

```

Problemas frecuentes

¿Qué pasa si a la historia le falta información?

Un agente nuevo no aparece

La suite web falla con un viewport de 0x0 o sin URL

```
npm test
```

corre 0 escenarios

Un escenario nunca se ejecuta

"No encuentro k6"

```
npm run perf
```

no corre un perfil con carga desde el agente

Una carpeta de Xray/QMetry/AIO aparece como

```
C:/Program Files/Git/...
```

Un conector MCP no conecta

¿Los tests verifican la base de datos siempre?

Falta una librería de Python

Extender el proyecto

Agregar un agente

- 1 Creá el archivo**
`.claude/agents/mi-agente.md` con frontmatter `name` y `description`. La descripción es lo que usa Claude Code para decidir cuándo invocarlo.
- 2 Escribí el cuerpo**
 En español: rol, entradas, proceso, salida y reglas.
- 3 Definí el formato**
 Si genera un formato propio, sumá su plantilla en `plantillas/` o su script en `scripts/`, y su carpeta en `output/`.
- 4 Normalizá las tablas**
 Si genera tablas en un `.md`, pasalas por `scripts/formatear_tablas.py`.

Skills, conectores y herramientas

Skill	Conector MCP	Herramienta
Una carpeta en <code>.claude/skills/</code> con su <code>SKILL.md</code> ; se carga solo cuando hace falta.	Una entrada en <code>.mcp.json</code> , sin secretos; los tokens se leen del <code>.env</code> con un script de <code>scripts/mcp/</code> .	Una subcarpeta en <code>herramientas/</code> con su README.

Regenerar esta documentación

```
node docs/generar.js # diagramas (PNG) y PDF, con el Playwright de k0lmena
```

El sitio es `docs/index.html` (publicado con GitHub Pages en underc0delabs.github.io/k0lmenIA); el PDF sale del mismo HTML con estilos de impresión, y los diagramas de `docs/diagramas/`.

Licencia

Licencia MIT — © 2026 QARMY. k0lmena es un proyecto open source de Danilo Vezzoni. Hecho por la comunidad de Underc0de.